Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #3| Ejecución de Datos en el Programa

Ingeniería en Desarrollo de Software



TUTOR: Felix Acosta Hernandez

ALUMNO: Edgar Enrique Cuamea Ochoa

FECHA: 06 de Abril del 2024

Contenido

Introducción. 3

Descripcion. 4

Justificación. 5

Desarrollo. 6

Conclucion. 11

Referencias. 12

Introducción.

En esta actividad veremos conceptos básicos sobre la programación orientada a objetos tales como las herencias simples o herencias múltiples, así como declarar la clase y los atributos de los diferentes objetos, por lo que es necesario graficar el modelo de entidad relación, con nuestra base de datos anteriormente creada, realizaremos una conexión con la base de datos así como poder realizar tablas a través de código en consola por lo que tendremos que tener instalado los programas de visual studio así como el lenguaje de C++ ya que es el lenguaje que estaremos utilizando para la conexión con la base de datos además de utilizar ODCB, un driver que nos permite conectarnos a una base de datos utilizando el lenguaje C++ por medio de la consola en visual studio, este driver es útil ya que lo utilizaremos para conectarnos a las bases de datos, aunque tengamos nuestra base de datos en local, ya que este driver es necesario para la conexión con las bases de datos, crearemos nuestra conexión y llamaremos información desde la base de datos para construir nuestras tablas.

Descripcion.

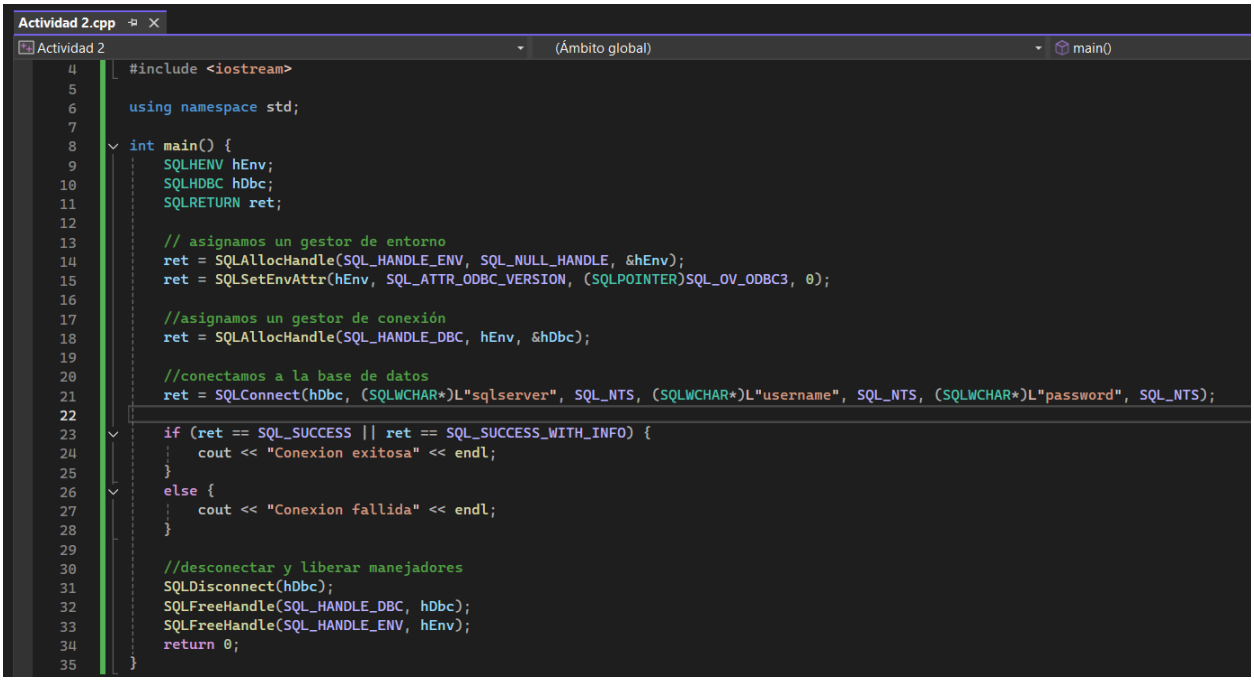
Realizaremos una base de datos en Microsoft SQL server para la realización de la actividad pues tendremos que analizar un contexto donde se nos indicara cuales clases y atributos tendremos que realizar por lo que es necesario descargar e instalar Microsoft SQL server para la realización de esta actividad, una vez hecha la base de datos, descargaremos un driver para podernos conectar a la base de datos por medio de una consola en visual studio por lo que es necesario instalar driver de OBDC, ya que este es necesario para podernos conectar a la base de datos ya que sin este driver, no podremos realizar la actividad por lo que una vez instalado y configurado procederemos a realizar una conexión a la base de datos para posteriormente realizar algunas tablas donde podremos algunos datos que consultaremos más adelante, realizaremos capturas del proceso de la actividad, desde el código ejecutado para seleccionar la información de las tablas así como la información que se imprimirá en la consola ya que realizaremos tablas con esta información que estaremos recopilando del servidor por medio de Código

Justificación.

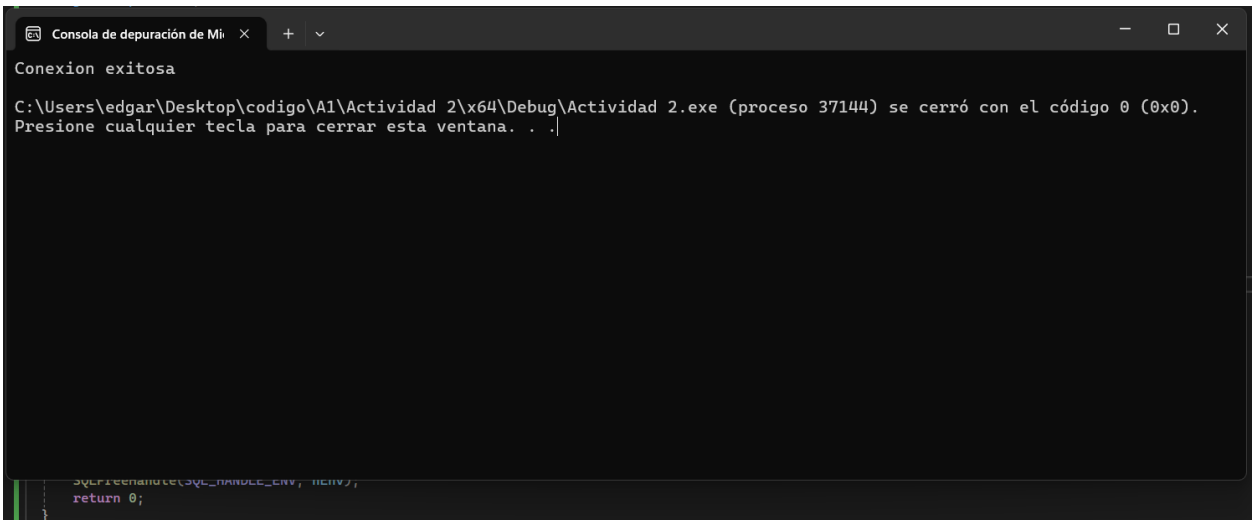
Utilizaremos el programa de Microsoft SQL Server para la creación de nuestra base de datos ya que este programa es bastante conocido para la gestión de las bases de datos además de ofrecer herramientas integradas que nos simplifican las administración de estas bases de datos además de proporcionar un ecosistema de Microsoft ya que funcionan con diferentes programas de esta compañía por lo que la seguridad de este programa es robusta con un buen rendimiento ya que maneja un volumen alto de datos de manera eficiente además de que el lenguaje SQL es un estándar para la creación de bases de datos por lo que podremos aplicarla al crear una base de datos local así como realizar sistemas donde se necesite una gestión de datos, utilizaremos el driver de OBDC para realizar la conexión con la base de datos por medio de una consola en visual studio por lo que utilizaremos código en visual studio para realizar la conexión así como el realizar tablas con la información obtenida desde la base de datos creada en la actividad 1

Desarrollo.

Abrimos nuestro proyecto en visual studio para realizar nuestra conexión a la base de datos,



Una vez que incluidas las librerias pasa la utilizacion de las funciones de sql, realizamos una concexion



verificando que tenemos una conexión exitosa, agregamos un encabezado de los datos que queremos imprimir en pantalla así como datos que traeremos desde el servidor de SQL

```
// Nos conectamos a la base de datos
ret = SQLConnect(hDbc, (SQLWCHAR*)L"sqlserver", SQL_NTS, (SQLWCHAR*)L"Username", SQL_NTS, (SQLWCHAR*)L"Password", SQL_NTS);

if (ret == SQL_SUCCESS || ret == SQL_SUCCESS_WITH_INFO) {
    cout << "Conectado a la base de datos exitosamente." << endl;

    // Ejecución de una consulta
    SQLHSTMT hStmt;
    ret = SQLAllocHandle(SQL_HANDLE_STMT, hDbc, &hStmt);

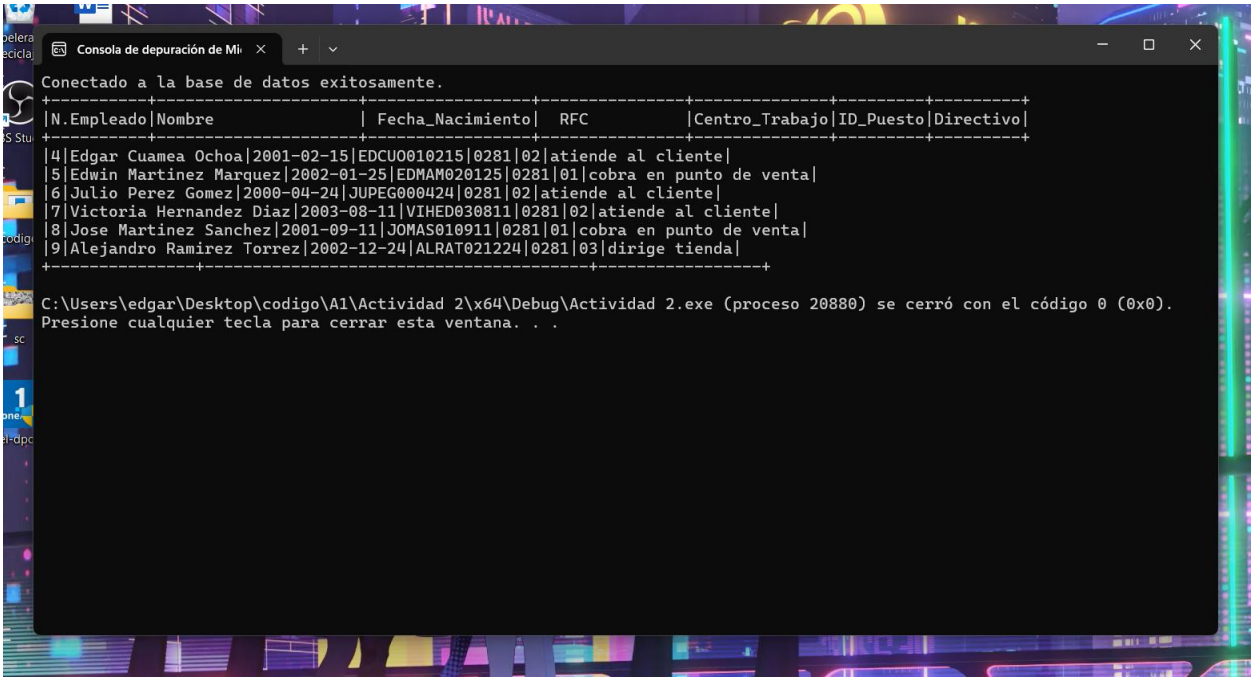
    // Imprimir encabezados de la tabla
    cout << "+-----------+--------------------+-----------------+---------------+--------------+---------+---------+" << endl;
    cout << "|N.Empleado|Nombre               | Fecha_Nacimiento|  RFC          |Centro_Trabajo|ID_Puesto|Directivo|" << endl;
    cout << "+-----------+--------------------+-----------------+---------------+--------------+---------+---------+" << endl;

    // Ejemplo de consulta SELECT
    ret = SQLExecDirect(hStmt, (SQLWCHAR*)L"SELECT * FROM Empleados", SQL_NTS);
    if (ret == SQL_SUCCESS || ret == SQL_SUCCESS_WITH_INFO) {
        int Num_Empleado;
        SQLCHAR Nombre[50];
        SQLCHAR Apellido_Materno[50];
        SQLCHAR Apellido_Paterno[50];
        SQLCHAR Fecha_Nacimiento[50];
        SQLCHAR RFC[50];
        SQLCHAR Centro_Trabajo[50];
        SQLCHAR ID_Puesto[50];
        SQLCHAR Directivo[50];
        while (SQLFetch(hStmt) == SQL_SUCCESS) {
            SQLGetData(hStmt, 1, SQL_C_LONG, &Num_Empleado, 0, NULL);
            SQLGetData(hStmt, 2, SQL_C_CHAR, Nombre, sizeof(Nombre), NULL);
            SQLGetData(hStmt, 3, SQL_C_CHAR, Apellido_Paterno, sizeof(Apellido_Paterno), NULL);
            SQLGetData(hStmt, 4, SQL_C_CHAR, Apellido_Materno, sizeof(Apellido_Materno), NULL);
            SQLGetData(hStmt, 5, SQL_C_CHAR, Fecha_Nacimiento, sizeof(Fecha_Nacimiento), NULL);
            SQLGetData(hStmt, 6, SQL_C_CHAR, RFC, sizeof(RFC), NULL);
            SQLGetData(hStmt, 7, SQL_C_CHAR, Centro_Trabajo, sizeof(Centro_Trabajo), NULL);
            SQLGetData(hStmt, 8, SQL_C_CHAR, ID_Puesto, sizeof(ID_Puesto), NULL);
            SQLGetData(hStmt, 9, SQL_C_CHAR, Directivo, sizeof(Directivo), NULL);
```

Realizamos nuestra consulta select con los datos que queremos así como el encabezado y la ultima fila para nuestra tabla.

```
57
58                  // Imprimir datos de la tabla
59                  cout << "|" << Num_Empleado << "|" << Nombre << " " << Apellido_Paterno << " " << Apellido_Materno << "|" << Fecha_Nacimiento << "|" << RFC << "|" << Centro_Trabajo << "|" << ID_Puesto << "|" << Directi
60              }
61
62              // Imprimir línea final de la tabla
63              cout << "+-----------+--------------------+-----------------+---------------+--------------+---------+---------+" << endl;
64
65              // Liberar el manejador de conexión
66              SQLFreeHandle(SQL_HANDLE_STMT, hStmt);
67          } else {
68              cout << "Fallo la ejecución de la consulta" << endl;
69          }
70      } else {
71          cout << "Fallo la conexion a la base de datos" << endl;
72      }
73
74      // Desconectar y liberar gestores de entorno
75      SQLDisconnect(hDbc);
76      SQLFreeHandle(SQL_HANDLE_DBC, hDbc);
77      SQLFreeHandle(SQL_HANDLE_ENV, hEnv);
78
79      return 0;
80  }
```

Imprimimos los datos de la tabla con los datos concatenados para tener los datos de un solo empleado en una sola fila, despues de consultar todos los datos de la tabla, desconectamos los puertos y liberamos los gestores del entorno por lo que imprime en la consola la siguiente informacion



Podemos ver que imprimio la información contenida en la base de datos, pero necesitamos tener una tabla visualmente mejor para poder diferenciar los datos de cada uno de los empleados correctamente ya que necesitamos que la tabla se vea alineada por lo que realizaremos unos cambios en el codigo para que esta tabla se vea bien,

```
    }
    //concatenamos nombre
    string Apellidos = string((char*)Apellido_Paterno) + " " + string((char*)Apellido_Materno);
    string NombreCompleto = string((char*)Nombre) + " " + string(Apellidos);
    string dir = string((char*)Directivo);
    string respuesta;

    if (dir == "1") {
        respuesta = "SI";
    }
    else
    {
        respuesta = "NO";
    }

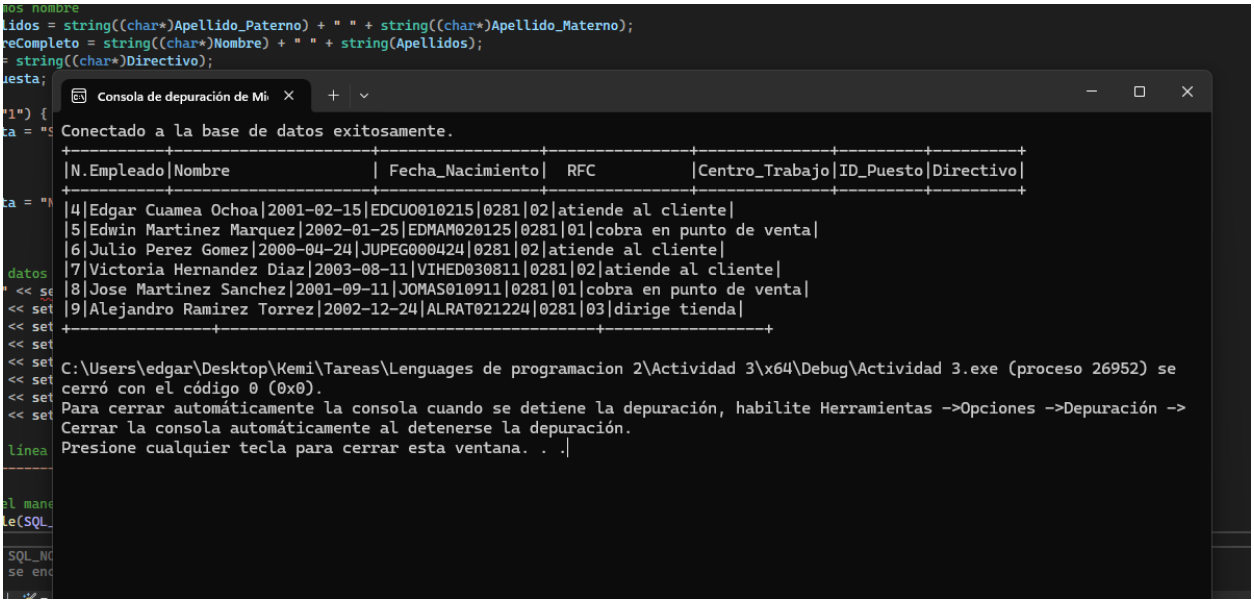

    // Imprimir datos de la fila con alineación
    cout << "| " << setw(10) << left << Num_Empleado
        << "| " << setw(30) << left << NombreCompleto
        << "| " << setw(10) << left << Fecha_Nacimiento
        << "| " << setw(15) << left << RFC
        << "| " << setw(15) << left << Centro_Trabajo
        << "| " << setw(9) << left << ID_Puesto
        << "| " << setw(1) << left << Directivo
        << "| " << setw(15) << left << respuesta << " |" << endl;

    // Imprimir línea final de la tabla
    cout << "+----------+------------------------------------------+----------------+-------------+-------------+---------+---------+" << endl;

    // Liberar el manejador de conexión
```

En el codigo anterior a este habiamos concatenado todos los datos de latabla para que aparecieran en una sola fila y poder construir la tabla por lo que en este caso concatenamos el nombre y los

apellidos para poder construir la tabla con delimitaciones, por lo que el nombre concatenado se lo asignamos a la variable NombreCompleto, despues realizamos una comparacion a la variable dir, donde este es igual a 1 o 0 nos dara un resultado de si o no por lo que despues realizamos una serie de pasos dodne imprimimos los datos de la fila con alineacion donde a cada una de las variables les asignamos un numero maximo de caracteres por lo que el numero de empleado se asignamos 10 para que así quede con la tabla.



Al ejecutar el codigo, este se sigue viendo igual, ya que aunque este bien escrito el codigo, olvidamos agregar una biblioteca <iomanip> que nos permite controlar datos de entrada y salida como el ancho del campo para poder alinear las columnas en una tabla

```
//concatenamos nombre
string Apellidos = string((char*)Apellido_Paterno) + " " + string((char*)Apellido_Materno);
string NombreCompleto = string((char*)Nombre) + " " + string(Apellidos);
string dir = string((char*)Directivo);
string respuesta;

if (dir == "1") {
    respuesta = "SI";
}
else
{
    respuesta = "NO";
}


// Imprimir datos de la fila con alineación
cout << "| " << setw(9) << left << Num_Empleado
    << "| " << setw(28) << left << NombreCompleto
    << "| " << setw(16) << left << Fecha_Nacimiento
    << "| " << setw(13) << left << RFC
    << "| " << setw(13) << left << Centro_Trabajo
    << "| " << setw(8) << left << ID_Puesto
    << "| " << setw(1) << left << Directivo
    << "| " << setw(4) << left << respuesta << " |" << endl;
}
```

Al modificar un poco los caracteres que puede tener nuestra tabla contando los espacios vacios y ejecutar el codigo nos muestra la siguiente tabla:

```
Consola de depuración de Mi
Conectado a la base de datos exitosamente.
+-----------+-----------------------------+-----------------+--------------+--------------+---------+----------+
|N.Empleado|Nombre                        | FechaNacimiento |  RFC         |Centro_Trabajo|ID_Puesto|Directivo|
+-----------+-----------------------------+-----------------+--------------+--------------+---------+----------+
| 4         | Edgar Cuamea Ochoa          | 2001-02-15      | EDCUO010215  | 0281         | 02      | 0| NO    |
| 5         | Edwin Martinez Marquez      | 2002-01-25      | EDMAM020125  | 0281         | 01      | 0| NO    |
| 6         | Julio Perez Gomez           | 2000-04-24      | JUPEG000424  | 0281         | 02      | 0| NO    |
| 7         | Victoria Hernandez Diaz     | 2003-08-11      | VIHED030811  | 0281         | 02      | 0| NO    |
| 8         | Jose Martinez Sanchez       | 2001-09-11      | JOMAS010911  | 0281         | 01      | 0| NO    |
| 9         | Alejandro Ramirez Torrez    | 2002-12-24      | ALRAT021224  | 0281         | 03      | 1| SI    |
+-----------+-----------------------------+-----------------+--------------+--------------+---------+----------+

C:\Users\edgar\Desktop\Kemi\Tareas\Lenguages de programacion 2\Actividad 3\x64\Debug\Actividad 3.exe (proceso 1244) se c
erró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

Cada uno de los datos de los empleados estan correctamente alineados, por lo que podemos conculir que la consulta con alineacion se realizo correctamete, el unico problema que veo es que la RFC la calcula de manera incorrecta ya que solo obtiene datos de la base de datos por lo que sabemos que el problema esta en la base de datos ya que no se calculo bien desde la generacion de el empleado por lo que se podria implementar on codigo que lo calcule nuevamente o simplemente arreglarlo desde la base de datos ya que este no guardo la información correctamente

Conclucion.

Con la realización de la base de datos Act 1 en SQL server management y el driver de ODBC, realizamos una consulta de la información que tenemos guardada en nuestra base de datos utilizando las librerías sql.h y sqlext.h que nos otorgaran funciones para la conexión con las bases de datos por lo que realizamos una consulta utilizando la consola utilizando código C++, utilizamos un código estándar para conectarnos con la base de datos y realizamos una consulta select por lo que en ese punto podremos especificar qué información queremos y como queremos que lo muestre en la consola, utilizamos la librería iomaniop, esta nos otorga la manipulación de datos de entrada y salida por lo que realizamos nuestra tabla con signos y los alineamos utilizando esta librería, realizamos nuestra tabla con los datos que obtuvimos de la base de datos así como poder manipular los datos para saber cuantos espacios podemos otorgar a diferentes filas así para construir la tabla correctamente sin errores y bien alineada.

Referencias.

Hackemi. (s. f.-a). *GitHub - Hackemi/Lenguajes_Prog2*. GitHub. https://github.com/Hackemi/Lenguajes_Prog2

FelixTEOPM. (s. f.). *ConexionBD/ConexionBD/ConexionBD.cpp at master · FelixTEOPM/ConexionBD*. GitHub. https://github.com/FelixTEOPM/ConexionBD/blob/master/ConexionBD/ConexionBD.cpp